



Livello di rete

Parte 2

Internet, Web e Cloud
Claudia Canali

Due funzioni chiave



1) Inoltro (forwarding)

Quando un router riceve un pacchetto lo deve trasferire sull'appropriato collegamento di uscita

Next hop: inoltro al router successivo sul percorso verso destinazione

1) Instradamento (routing)

Il livello di rete deve determinare il percorso che i pacchetti devono seguire tramite **algoritmi di instradamento (algoritmi di routing)**

In letteratura i termini inoltro e instradamento sono a volte considerati **sinonimi**: noi li **distingueremo** utilizzandoli in maniera precisa

Inoltro → azione locale sul router da interfaccia di ingresso a uscita

Instradamento → processo globale che determina intero percorso

Analogia con un viaggio in auto

Inoltro → andare attraverso un singolo svincolo (es. una rotonda a più uscite)

Instradamento → pianificare l'intero percorso

Piano dei dati e piano di controllo

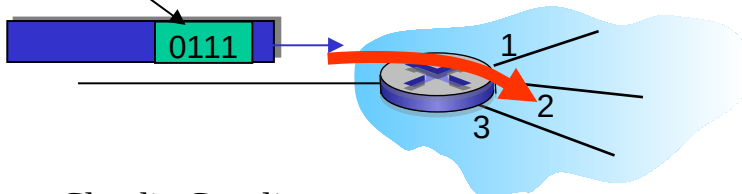


UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Piano dei dati

- Funzioni a livello di singolo router
- Determina come un datagramma in arrivo su una porta di ingresso passi sulla porta di output (*inoltro*)
- Funzione molto veloce (nanosecondi)
- Solitamente implementata in hardware

Pacchetto in input



Piano di controllo

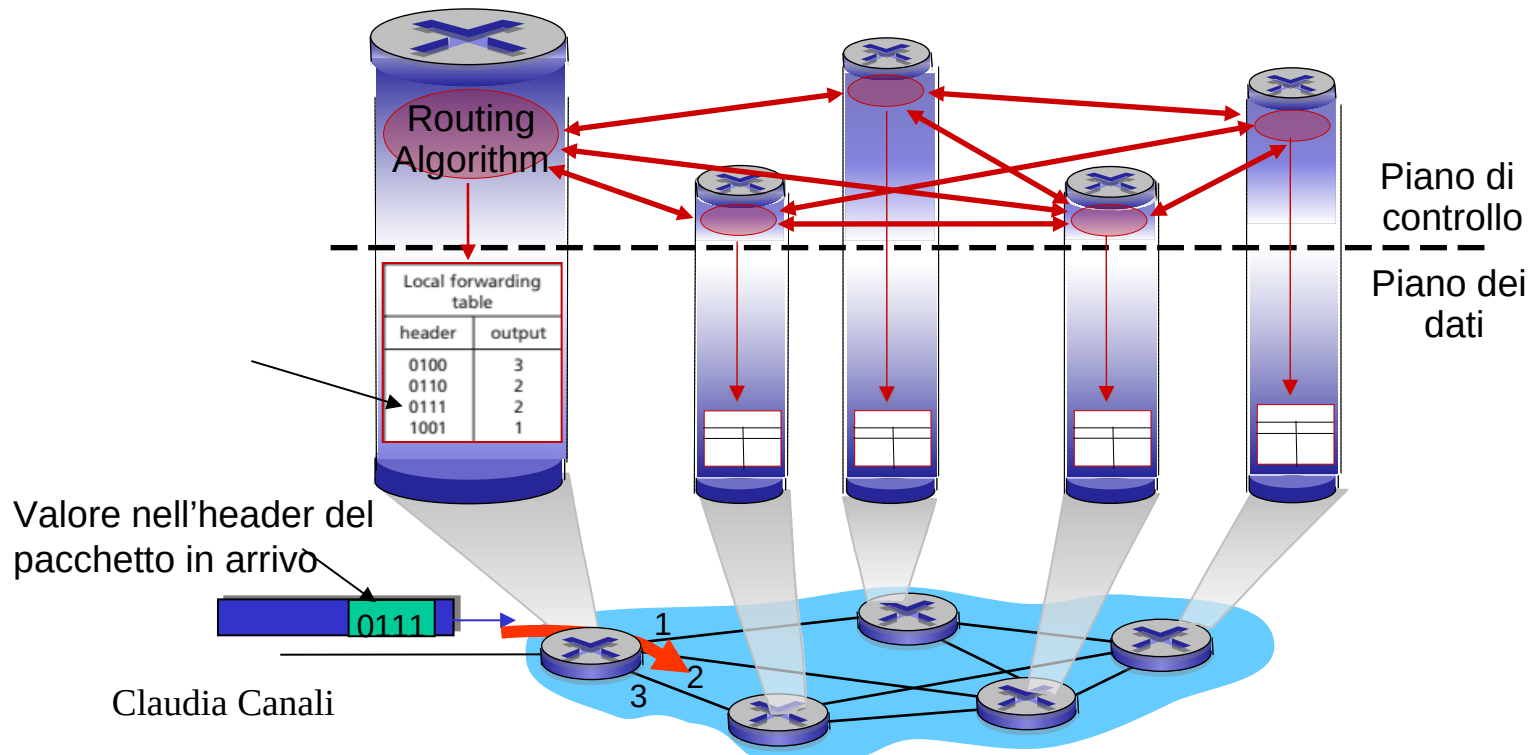
- Logica a livello di intera rete
- Determina come un datagramma viene instradato tra i router lungo il percorso end-end path da sorgente a destinazione
- Scala temporale più grande (secondi) – implementata in software
- Due approcci:
 - *Tradizionale: algoritmi di routing* implementati in ogni router
 - *Software-defined networking (SDN)*: implementato in server remoti (es. collocati in un data center remoto)

Tabella di inoltra

Elemento chiave di ogni router: **tabella di inoltra (forwarding table)**

I router estraggono dall'**header dei pacchetti in arrivo** valori che usano come **indice nella tabella** per capire a quale **interfaccia di uscita** il pacchetto deve essere diretto

Es. in figura: inoltra su interfaccia 2

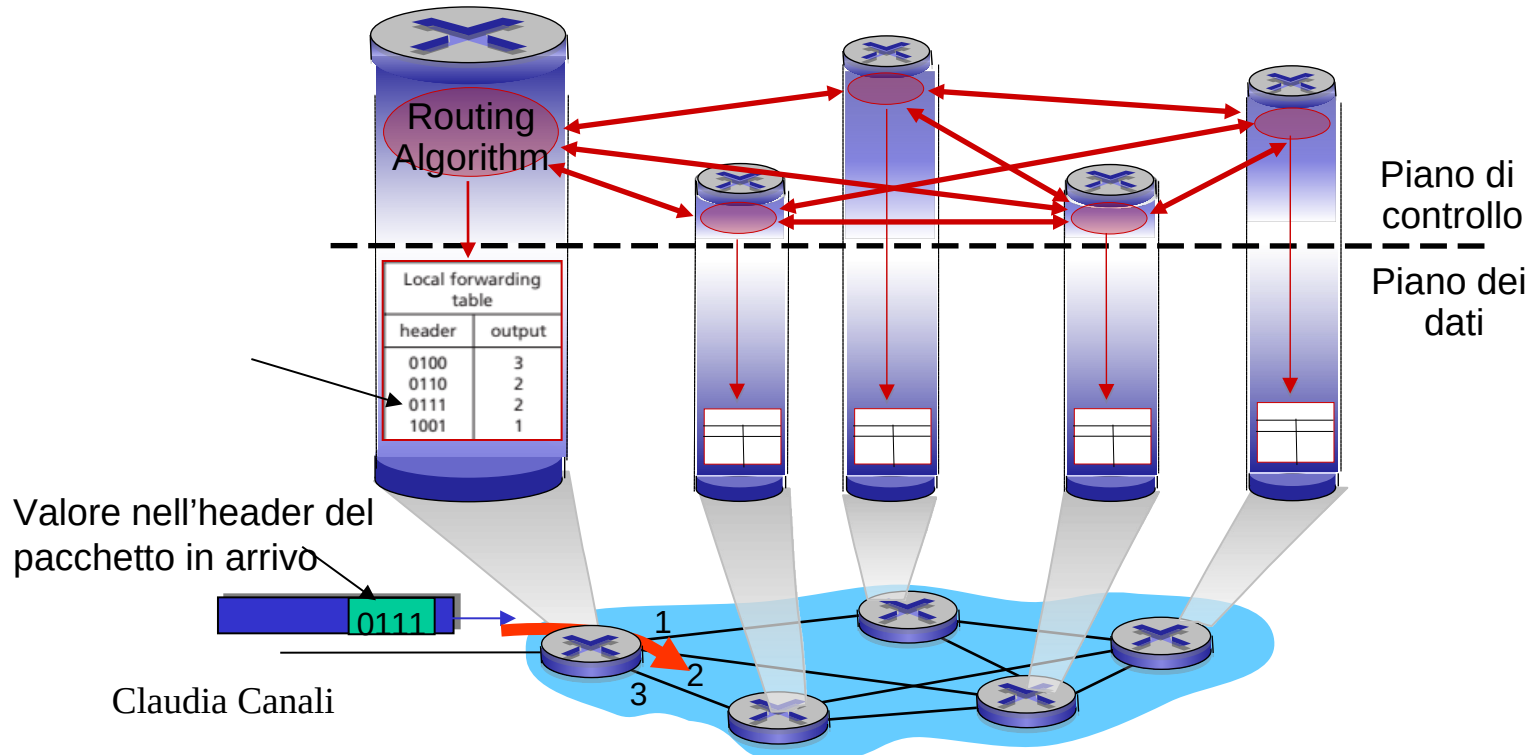


Approccio tradizionale

Come viene configurata la **tabella di inoltra**? L'**algoritmo di instradamento** determina i valori inseriti nelle tabelle di inoltra dei router

Approccio tradizionale: il router svolge sia funzione di inoltra che di instradamento → algoritmo di routing è implementato in ogni router

Routing basato su comunicazioni scambiate tra router (*vedremo più avanti...*)



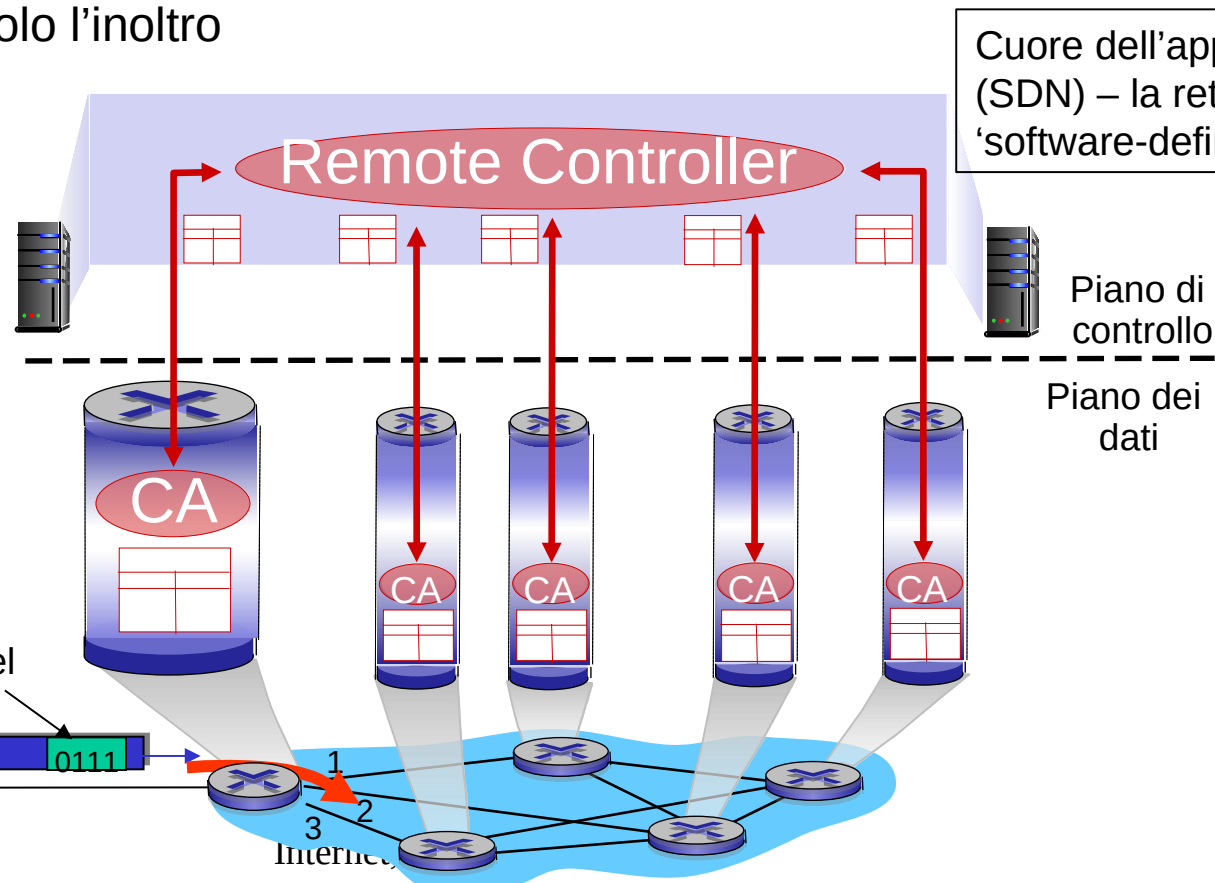
Approccio SDN

Un **controller** separato dai router (tipicamente remoto) interagisce con i **local control agents (CAs)**

Il controller calcola le tabelle di inoltro e le distribuisce ai router

Il router effettua solo l'inoltro

Controller remoto:
può essere
implementato in un
data center ad
elevata affidabilità e
ridondanza, gestito
da ISP o terza parte



Cuore dell'approccio
(SDN) – la rete è
'software-defined'

Piano di
controllo

Piano dei
dati

Valore nell'header del
pacchetto in arrivo

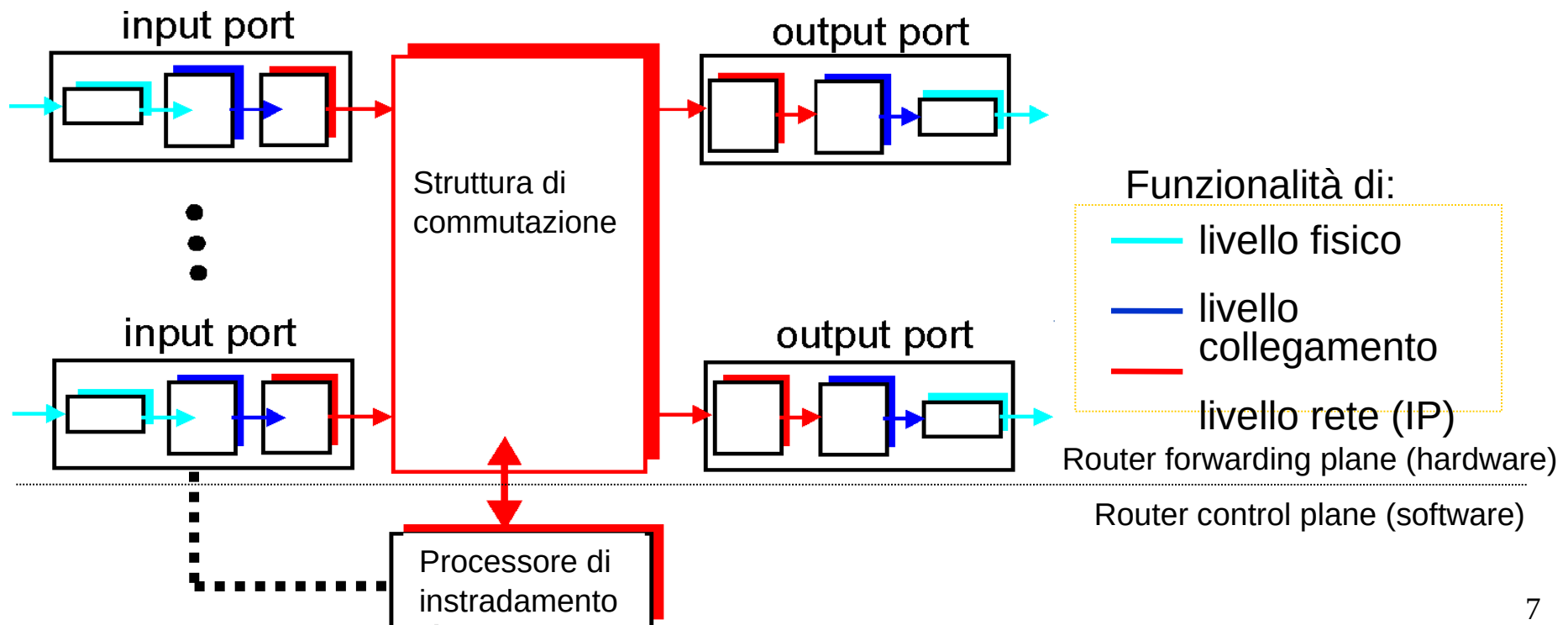
0111

Architettura di un router

4 componenti fondamentali nell'architettura di un router:

- Porta di ingresso
- Struttura di commutazione
- Processore di routing
- Porta di uscita

**Concentriamoci sulla
funzione di inoltra
dei router**



Architettura di un router

4 componenti fondamentali nell'architettura di un router:

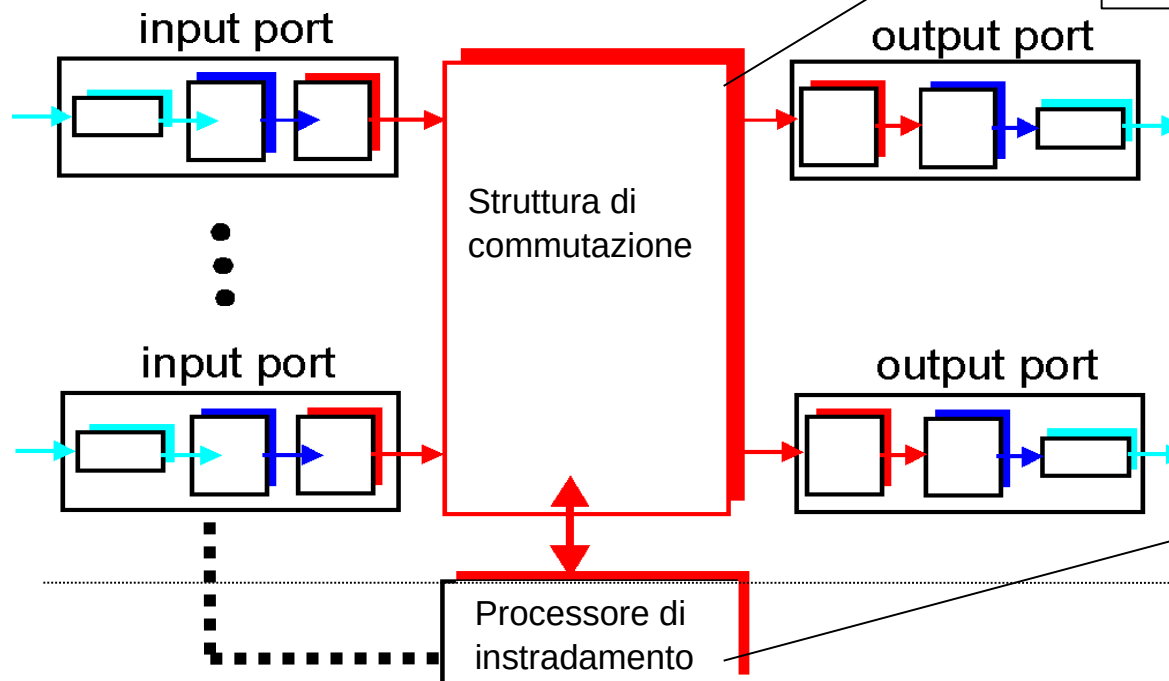
- **Porta di ingresso**
- **Struttura di commutazione**
- **Processore di routing**
- **Porta di uscita**

Connette fisicamente le porte di ingresso con quelle di uscita: una **vera e propria rete** contenuta in un router di rete!

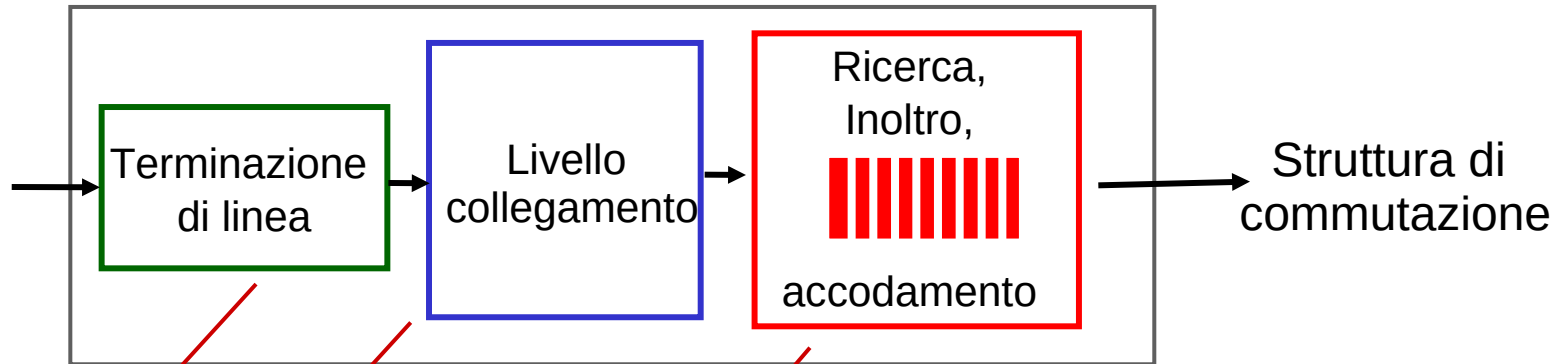
Esegue funzioni del piano di controllo
In **router tradizionali**, esegue algoritmo di instradamento
In **router SDN**, comunica con il controller remoto, riceve tabelle di inoltro e le installa

Router forwarding plane (hardware)

Router control plane (software)



Porta di input



Livello fisico

Ricezione a livello di bit

Livello di collegamento

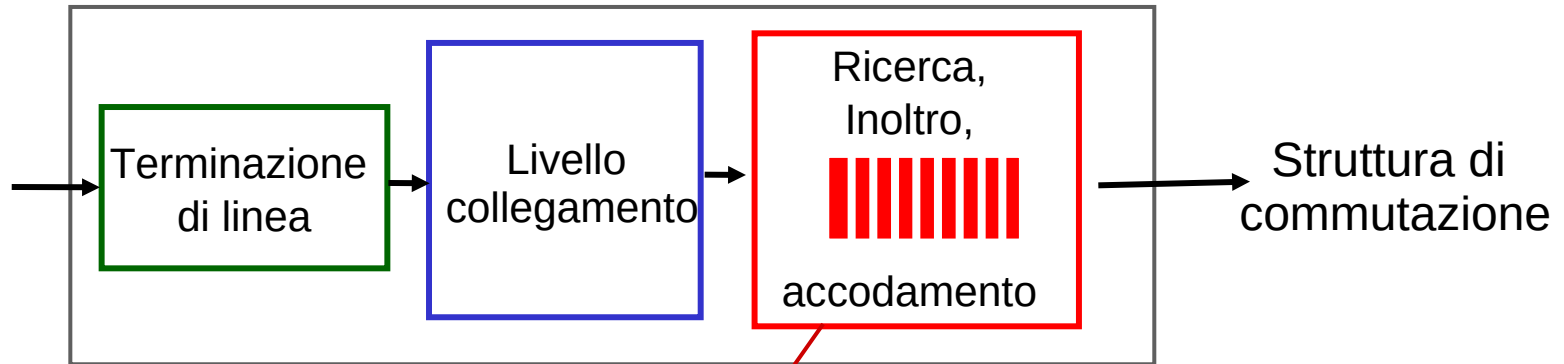
(Elaborazione a livello di collegamento – protocollo di decapsulamento)

Livello rete

- Porta di ingresso – funzionalità centrale per il router
- Usa valori dell'header del datagramma, **ricerca** porta di uscita nella forwarding table (**porte** = interfacce fisiche di input e output del router → non porte sw come quelle associate alle socket!)
- **Pacchetti di controllo** (info su protocollo di instradamento) inoltrati al **processore di instradamento**
- **Accodamento**: se i datagrammi arrivano più veloci dello switching rate della struttura di commutazione

Numero di porte: da relativamente poche su router aziendale a centinaia di porte a 10 Gbps in router ISP (Es. router Juniper MX2020 960 porte Ethernet a 10Gbps per totale capacità di 80 Tbps)

Porta di input



NOTA: porte di ingresso, struttura di commutazione e porte di uscita sono **implementate in hardware**, → costruite con **circuiti elettronici dedicati** (es. *ASIC Application-Specific Integrated Circuits* o *FPGA Field Programmable Gate Arrays*), e non tramite sw generale su CPU tradizionali. Es. link input 10 Gbps e datagramma di 64 byte → porta di input ha circa 51,2 ns per elaborare prima dell'arrivo del nuovo datagramma

Velocità impossibili per una implementazione software

- **Tabella di inoltro copiata sulla scheda di collegamento di ogni porta di ingresso** (decisione relativa presa dalle porte di ingresso senza invocare il processore di instradamento)
- **Inoltro basato sulla destinazione:** inoltro basato soltanto sull'indirizzo IP di destinazione (tradizionale)
- **Inoltro generalizzato:** inoltro basato su un insieme di campi dell'header del datagramma IP (qualunque numero di input può essere usato)

Struttura di commutazione

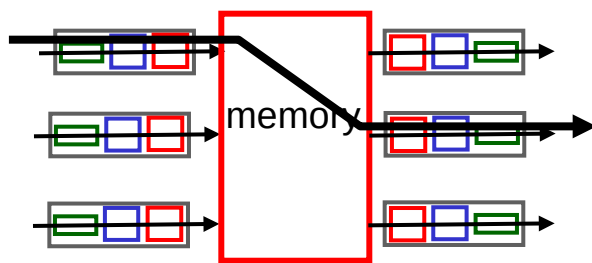


Detto anche Switching fabric – cuore del router

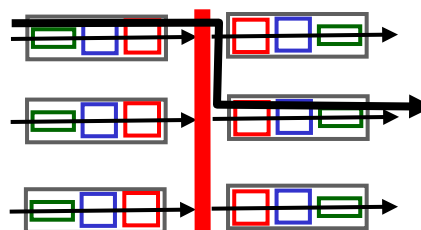
Funzione: spostamento del pacchetto dalla porta di ingresso a quella di uscita “opportuna”

Switching rate: frequenza a cui i datagrammi sono trasferiti da input a output

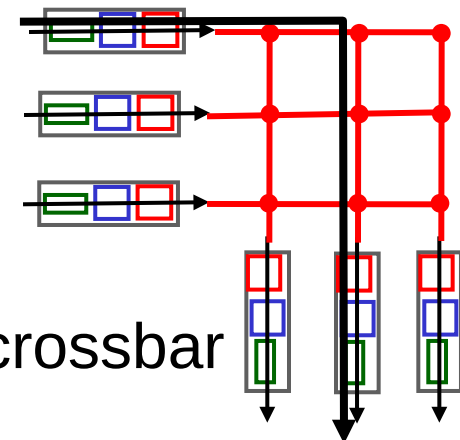
3 tecniche possibili: Commutazione basata su memoria (old), bus o rete di interconnessione crossbar



memoria



bus



crossbar

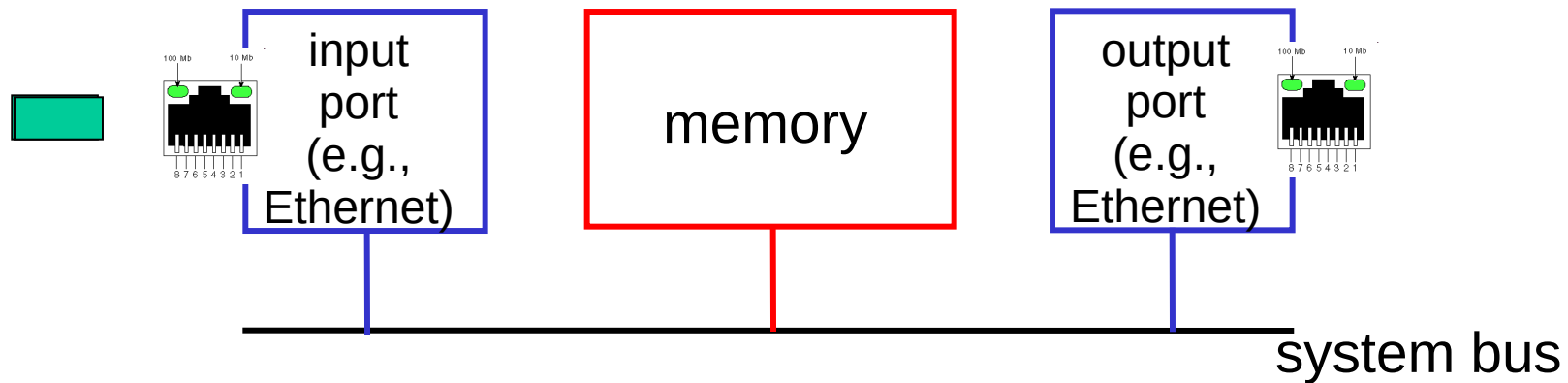
Memoria

Router di prima generazione – in genere erano calcolatori tradizionali

Commutazione sotto il **controllo della CPU** (processore di instradamento) – porte di ingresso e uscita come tradizionali dispositivi di I/O

Pacchetti in arrivo copiati in memoria e processati per individuare la porta di uscita

Velocità limitata dall'ampiezza di banda della memoria: se tale da poter scrivere o leggere B pacchetti al secondo → switching rate limitato a $B/2$



Bus

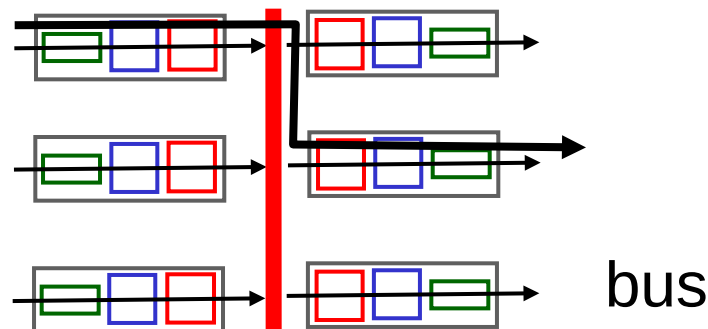
Datagrammi trasferiti **direttamente alle porte di uscita tramite un bus condiviso** senza intervento del processore di instradamento

Etichetta interna di commutazione aggiunta al pacchetto che indica la porta di output

Il pacchetto viene ricevuto da tutte le porte di output ma solo quella che corrisponde all'etichetta lo raccoglierà

Larghezza di banda della commutazione **limitata da quella del bus** (sul bus si trasferisce un datagramma alla volta – **bus contention**)

Switch Cisco 5600 → bus a 32 Gbps (router aziendale)



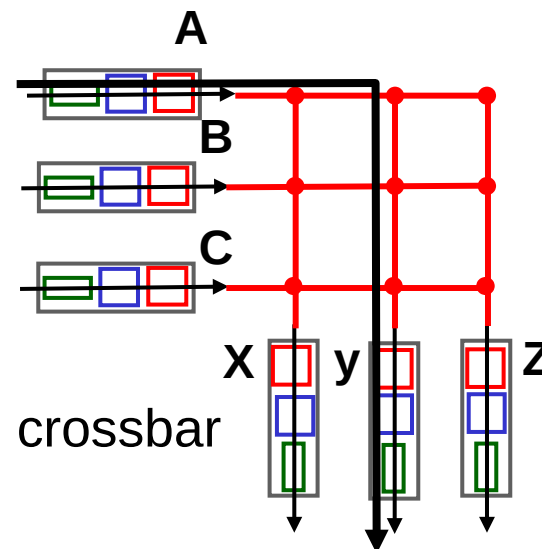
Uso di rete di interconnessione più sofisticata

Matrice di commutazione (crossbar switch): $2n$ bus che collegano n porte di ingresso con n porte di uscita - ogni punto di incrocio aperto o chiuso

Un **controller** (la cui logica è parte stessa della struttura) gestisce gli incroci per inoltrare il datagramma sulla porta giusta

Switch Cisco 12000 usa tecnica crossbar → 60 Gbps

NOTA: mentre un datagramma da A viene inoltrato a Y, un altro può essere inoltrato da B a X perchè usano bus di input e output differenti
--> inoltro di più pacchetti in parallelo
Matrice di commutazione **non-blocking**

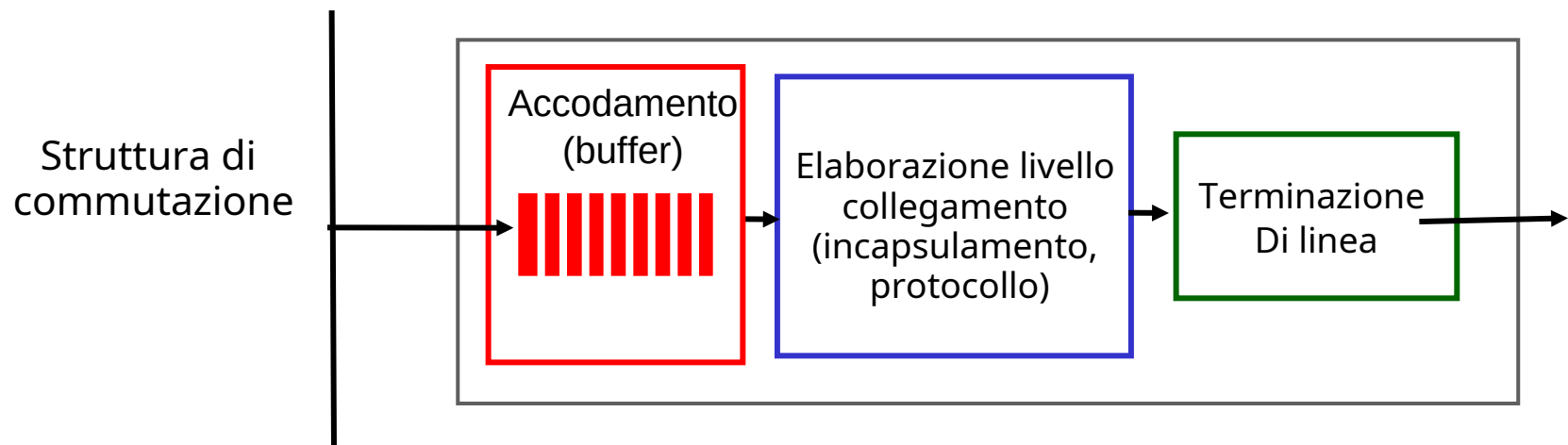


Componente importante!

Buffering necessario quando i datagrammi arrivano dalla struttura di commutazione a una frequenza più alta del **rate di trasmissione in output**

Quando ci sono più pacchetti accodati sulle porte di uscita uno **schedulatore di pacchetti (packet scheduler)** deve stabilire in quale ordine trasmettere i pacchetti in coda

- **Priority scheduling** – può essere implementato un meccanismo di priorità per avere pacchetti che godranno di performance migliori



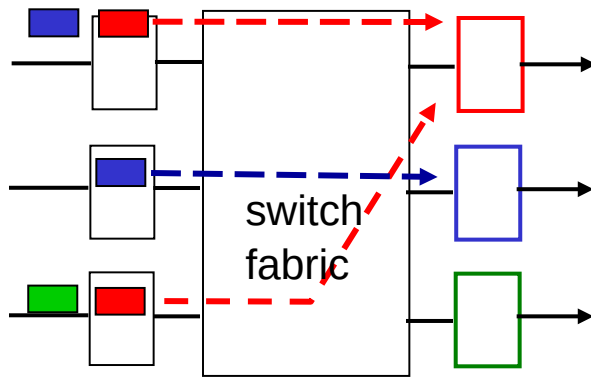
Dove si verifica l'accodamento?



- Può verificarsi sia alle porte di ingresso che di uscita
- Luogo e lunghezza delle code dipende da
 - Traffico di rete
 - Velocità relative della struttura di commutazione e della linea
- Quando queste code crescono → la memoria nei router può esaurirsi e può verificarsi **perdita di pacchetti**
- **Ipotesi**
 - Linea di input e output: stesso tasso di trasmissione R_{line} / sec
 - N porte di input e N di output
 - Switching rate R_{switch} (tasso trasferimento pacchetti da porte di input a porte di output)
 - **Se $R_{\text{switch}} \sim N \times R_{\text{line}}$ l'accodamento alle porte di input sarà trascurabile ... ma questo è sufficiente?**

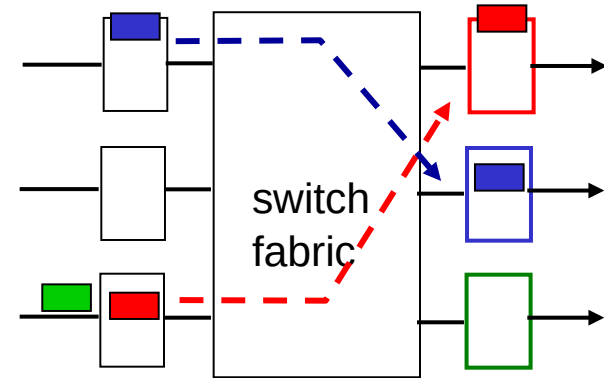
Accodamento in ingresso

- Cosa avviene se la struttura di commutazione non riesce a trasferire *tutti* i pacchetti in arrivo?
- **Input buffer overflow**



Contesa sulla porta di uscita

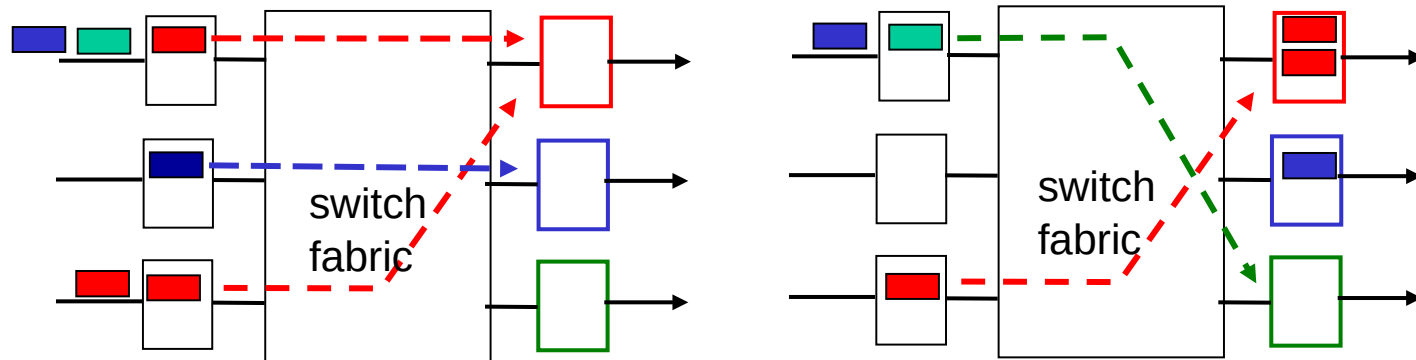
due pacchetti in testa alle code di ingresso destinati alla stessa coda di uscita: solo un datagramma rosso può essere trasferito (assumiamo quello superiore → datagramma rosso inferiore bloccato)



Un “tempo di trasmissione di pacchetto” più tardi: il pacchetto verde è rimasto bloccato anche se destinato a coda di output libera) → **HOL (Head Of Line) blocking**

Accodamento in uscita

- Cosa può avvenire alla porta di uscita?
- Se la frequenza di arrivo dalla struttura di commutazione eccede la velocità di trasmissione sulla linea di uscita
 - **Output buffer overflow**
 - **Ritardo in coda o perdita di pacchetti!**



N pacchetti destinati alla stessa porta di uscita

Nel tempo richiesto per l'invio di un pacchetto sulla linea di uscita, N pacchetti arrivano alla stessa porta (uno da ogni porta di ingresso) che si dovranno mettere in coda

E potrebbe succedere lo stesso allo slot di tempo successivo – perchè la struttura di commutazione è N volte più veloce della linea in uscita

→ **esaurimento dello spazio di memoria sull'output buffer!**

Ma chi scarto?



- Bisogna decidere se scartare l'ultimo pacchetto arrivato (politica **drop-tail**) o rimuoverne uno già in coda
- Può essere vantaggioso eliminare un pacchetto già in coda prima che il buffer sia pieno per dare il prima possibile al mittente un segnale di congestione
- Proposte numerose tecniche di eliminazione e marcatura dei pacchetti (**active queue management – AQM**)
- Inoltre quando ci sono più pacchetti in coda → **scheduler di pacchetti** deve stabilire in quale ordine trasmetterli
- Quanto spazio va dedicato al buffering?
- Per molti anni **regola empirica (rule of thumb)**: la dimensione del buffer doveva essere uguale al RTT medio (es. 150 ms) per la capacità del link C
 - $C = 10 \text{ Gbps link} \rightarrow B = \text{RTT} \times C = 1.5 \text{ Gbit buffer}$
- Raccomandazioni recenti: con N flussi TCP il buffer può essere

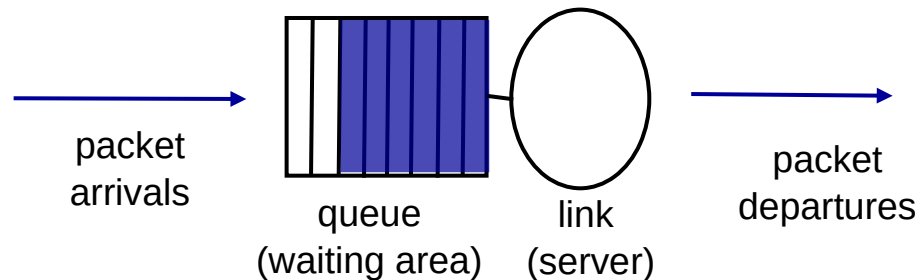
$$\frac{\text{RTT} \cdot C}{\sqrt{N}}$$

N può essere grande → diminuzione buffer significativa

Meccanismi di scheduling



- **Scheduler di pacchetti:** scelta della politica di invio sul link di uscita
- **FIFO (first in first out) scheduling:** invio nell'ordine di arrivo
- **Politica di scarto:** se un pacchetto arriva mentre la coda è piena chi scarto?
 - **tail drop:** scarto il pacchetto in arrivo
 - **priority:** scarto il pacchetto in base alle priorità
 - **random:** scarto un pacchetto random

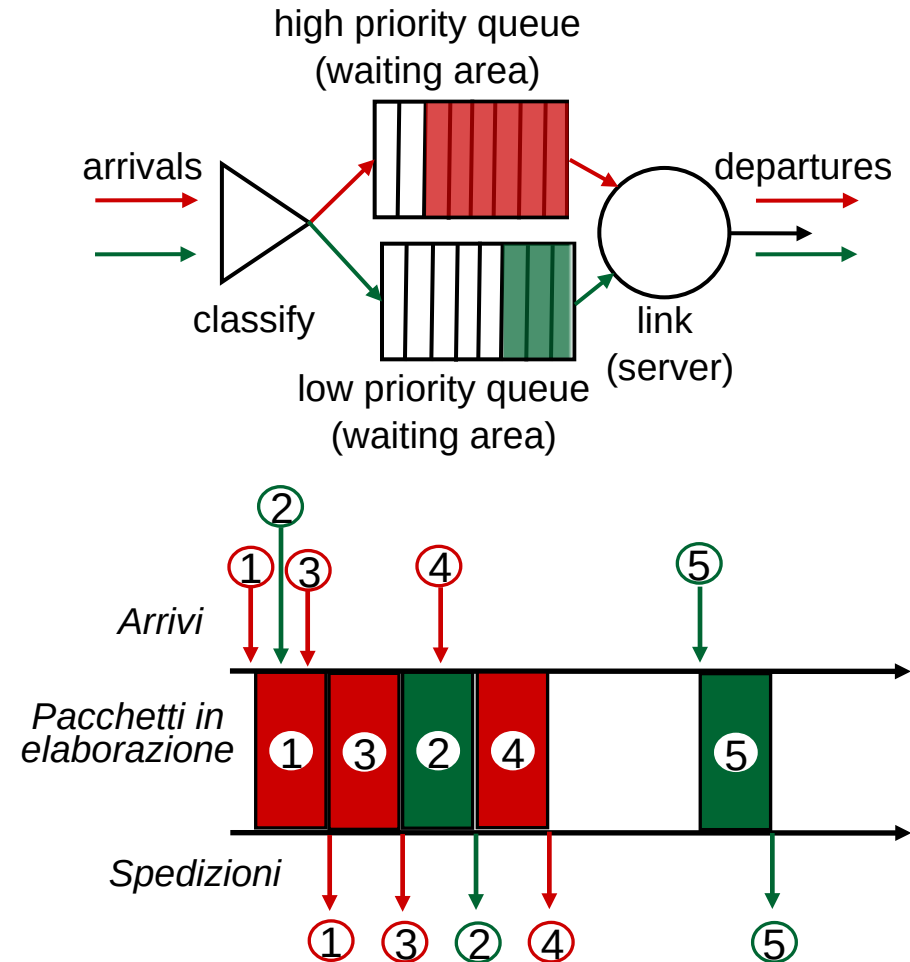


Meccanismi di scheduling con priorità



Code con priorità

- Mando il pacchetto con priorità più alta
- Più code con diverse priorità: ciascuna classe di traffico ha la sua coda
- La classe può dipendere da una marcatura esplicita o da altre informazioni contenute nell'header
- es- IP sorgente/destinazione, numeri di porta, protocollo, ...
- Es. dati di applicazioni voice-over-IP in tempo reale
- Politica FIFO dentro ogni coda

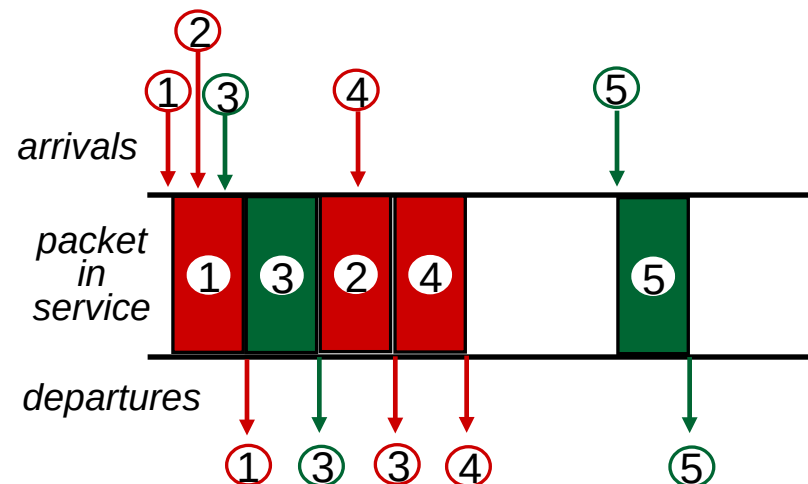


Meccanismi di scheduling: RR



- **Round Robin Scheduler (RR)**

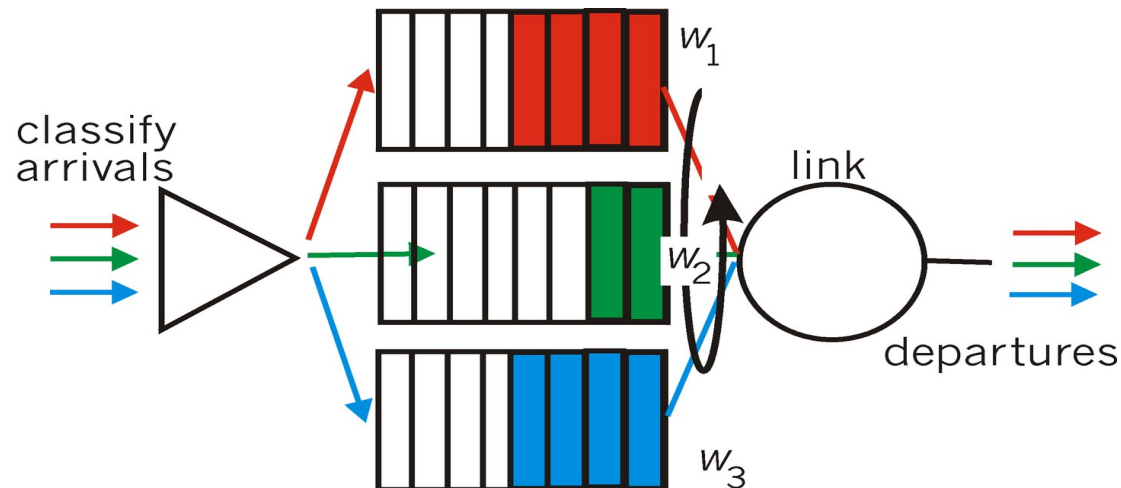
- Classi multiple, ma senza una rigida priorità di servizio
- Scandisce **ciclicamente** le classi, inviando un pacchetto da ognuna
- **Modalità conservativa**: il collegamento non resta mai inattivo finchè ci sono pacchetti da inviare → se una coda è vuota viene immediatamente scandita quella successiva



Meccanismi di scheduling: WFQ



- **Weighted Fair Queuing (WFQ)** - accodamento equo ponderato
 - Vasto impiego nei router
 - Pacchetti in arrivo classificati in base alle classi
 - Servizio ciclico, modalità conservativa
 - Diversamente dal Round Robin, ogni classe ottiene un servizio differenziato in base al **peso** w_i : in qualsiasi momento nella **coda** i ci siano pacchetti WFQ garantisce che ricevano una **frazione del servizio pari al peso della classe**
 - **Garanzia di una certa frazione della larghezza di banda**



Meccanismo di inoltro



- Consideriamo ora il cuore della **funzionalità di inoltro** del router: il meccanismo di **selezione della porta di uscita**
- Consideriamo il caso più semplice: **inoltro basato sulla destinazione**
- In base all'indirizzo IP di destinazione, deve essere identificata la porta (interfaccia) di uscita
- Supponiamo indirizzi a 32 bit (IPv4)
- Implementazione elementare: una riga per ogni possibile indirizzo
 - Impossibile: più di 4 miliardi di indirizzi
- Altra ipotesi: lavorare a 'intervalli di indirizzi'
- (Vedere figura successiva)

Meccanismo di inoltro



- **Ipotesi di router con 4 interfacce di collegamento in uscita**

<i>forwarding table</i>	
<u>Intervallo indirizzi di destinazione</u>	<u>Interfaccia</u>
da 11001000 00010111 00010000 00000000 a 11001000 00010111 00010111 11111111	0
da 11001000 00010111 00011000 00000000 a 11001000 00010111 00011000 11111111	1
da 11001000 00010111 00011001 00000000 a 11001000 00010111 00011111 11111111	2
<u>altrimenti</u>	3

Meccanismo di inoltro



- **Ipotesi di router con 4 interfacce di collegamento in uscita**

forwarding table

<u>Intervallo indirizzi di destinazione</u>	<u>Interfaccia</u>
da 11001000 00010111 00010000 00000000 a 11001000 00010111 00010111 11111111	0
da 11001000 00010111 00011000 00000000 a 11001000 00010111 00011000 11111111	1
da 11001000 00010111 00011001 00000000 a 11001000 00010111 00011111 11111111	2
<u>altrimenti</u>	3

**Potremmo ridurla
ancora usando
solo parte
dell'indirizzo
(prefisso)?
Lunghi quanto?**

Corrispondenza a prefisso più lungo



Il router **confronta l'indirizzo di destinazione con le righe nella tabella:**
se c'è corrispondenza inoltra all'interfaccia corrispondente

<u>Corrispondenza di prefisso</u>	<u>Interfaccia</u>
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

Es.

da **11001000 00010111 00010110 10100001** → 0

Da **11001000 00010111 00011000 10101010** → ?

Corrispondenza a prefisso più lungo



- *Longest Prefix Matching*
- Se un indirizzo corrisponde a più di una riga, il router adotta la regola di **corrispondenza a prefisso più lungo** → determina la corrispondenza più lunga e inoltra alla interfaccia associata
- Da **11001000 00010111 00011000 10101010** → 1
- La ricerca deve avvenire in **nanosecondi**: effettuata in **hardware** con tecniche che vanno **oltre la ricerca lineare** (algoritmi di ricerca veloci)
- Spesso usate Ternary Content Address Memories (TCAM)
 - Tipologie di memorie specializzate ad alta velocità in grado di ricercare nel loro intero contenuto in un singolo clock cycle
- Router Cisco arrivano a tabelle di inoltro con milioni di occorrenze

Esercizio



- Si consideri una rete con un router avente 4 collegamenti in uscita e che i pacchetti debbano essere inoltrati sulle interfacce di uscita come segue

Intervallo Indirizzi Destinazione		Interfaccia di collegamento
da	11100000 00000000 00000000 00000000	0
a	11100000 00111111 11111111 11111111	
da	11100000 01000000 00000000 00000000	1
a	11100000 01000000 11111111 11111111	
da	11100000 01000001 00000000 00000000	2
a	11100000 01111111 11111111 11111111	
	altrimenti	3

Scrivere una tabella di collegamento di 4 righe che usa il confronto a prefisso più lungo

Esercizio



- Descrivere come la vostra tabella di inoltra determini l'interfaccia di collegamento corretta per i datagrammi con i seguenti indirizzi

D1 11001000 10010001 01010001 01010101

D2 11100000 01000000 11000011 00111100

D3 11100000 01000001 00010001 01110111

"E' vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

E' inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia."